

Matr Studio — Trunk-Based Development & Monorepo Workflow

Oct 7, 2026 · @Joshua

1. Branching strategy

Matr Studio is one repository with one history. The waitlist, landing, user app and admin app all consume the same `packages/ui`, `packages/types` and `packages/api-client`, and the Hono Worker in `packages/backend` serves them all. Long-lived branches would drift apart on exactly those shared packages, so the repo follows **trunk-based development**: one branch, `main`, and short-lived branches that return to it quickly.

The rules

- `main` is the single source of truth. It is always releasable, because every merge to it deploys. Frontends go out as Cloudflare Workers static assets (or Pages, if you keep a Pages project) and the backend goes out as a Worker.
- Nobody commits to `main` directly. Every change arrives through a pull request (see section 3), and branch protection enforces it (section 4).
- Work branches are cut from the latest `main`, never from another feature branch.
- A branch lives for hours or a day or two, not weeks. If work is bigger than that, slice it into steps that can each merge safely, and hide the unfinished part behind a flag (section 6).
- There are no `develop`, `staging` or `release/*` branches. Environments are not branches. If a staging environment is added later, it is a second deploy target fed from `main`.

Branch names

A branch name is `<type>/<short-kebab-description>`. The type matches the Conventional Commit type the squashed commit will carry (section 2).

Prefix	Use for	Example
<code>feat/</code>	New user-facing or API capability	<code>feat/waitlist-referrals</code>

Prefix	Use for	Example
<code>fix/</code>	Bug fixes	<code>fix/api-cors-headers</code>
<code>chore/</code>	Maintenance, config, dependencies, migrations	<code>chore/d1-migrations</code>
<code>refactor/</code>	Internal restructuring with no behaviour change	<code>refactor/api-client-errors</code>
<code>docs/</code>	Documentation only	<code>docs/runbook-quota-alerts</code>

The first three prefixes are the core set. `refactor/` and `docs/` are optional additions that mirror commit types you will use anyway; drop them if you want a smaller vocabulary. Keep descriptions to a few words and put the app or package name in them when it helps (`feat/waitlist-referrals` , not `feat/referrals`), since the branch name is what you see in the PR list.

Daily loop

```
# start from an up-to-date trunk
git switch main
git pull --ff-only
git switch -c feat/waitlist-referrals

# commit small and often (conventions in section 2)
git add -p
git commit -m "feat(waitlist): add referral code field"

# keep the branch current by rebasing, not merging main into it
git fetch origin
git rebase origin/main

# publish and open the PR
git push -u origin feat/waitlist-referrals
gh pr create --fill --base main
```

Rebasing keeps the branch history linear and makes the eventual squash trivial. Because the PR is squash-merged, the individual commits on the branch are scaffolding: they help you and the reviewer while the PR is open and disappear into one commit on `main` afterwards.

Keeping `main` stable while merging often

Short branches only work if merging unfinished work is safe. Three habits make that true in this stack:

- **Flags over branches.** A half-built referral flow ships to `main` switched off, controlled by a `NEXT_PUBLIC_*` build variable or a value served by the API, and is turned on when it is ready.
- **Backward-compatible migrations.** A D1 migration must work with both the old and the new Worker code, because the schema and the code never deploy in the same instant. Section 6 gives the expand and contract pattern.
- **Small PRs.** A PR that touches one app or one package is reviewed in minutes, deploys one thing, and rolls back cleanly.

After the squash merge, delete the remote branch (enable *Automatically delete head branches* in repository settings) and the local one with `git branch -d`.

2. Commit conventions

Every commit that reaches `main` follows [Conventional Commits](#) with a **scope that names the directory** the change lives in. With squash and merge (section 3) the commit on `main` is the pull request title, so the rule is enforced on the PR title, and the commits inside the branch are free-form scaffolding.

Format

```
<type>(<scope>): <imperative summary, lower case, no full stop>
```

```
[optional body: why, not what]
```

```
[optional footer: BREAKING CHANGE: ..., Closes #12]
```

Example	Meaning
<code>feat(waitlist): implement email capture form</code>	New feature in <code>apps/waitlist</code>
<code>fix(backend): resolve D1 constraint error</code>	Bug fix in <code>packages/backend</code>

Example	Meaning
<code>chore(ui): update radix primitive buttons</code>	Maintenance of the shared component library in <code>packages/ui</code>
<code>refactor(repo): upgrade turbo to latest</code>	Root-level change that affects the whole workspace

Types

Type	Use for	Appears in changelog
<code>feat</code>	A new capability	Yes
<code>fix</code>	A bug fix	Yes
<code>perf</code>	A performance improvement	Yes
<code>refactor</code>	Restructuring without behaviour change	No
<code>chore</code>	Dependencies, config, migrations, tooling	No
<code>docs</code>	Documentation only	No
<code>test</code>	Adding or fixing tests only	No
<code>ci</code>	Workflow files under <code>.github/</code>	No

The changelog column is a convention you can adopt if you later generate release notes; nothing in this setup requires it.

Scopes

One scope per directory that can change independently. Keep the list closed so scopes stay meaningful, and enforce it in the PR title check (section 5).

Scope	Directory	Notes
<code>waitlist</code>	<code>apps/waitlist</code>	Pre-launch signup site
<code>landing</code>	<code>apps/landing</code>	Marketing site
<code>user-app</code>	<code>apps/user-app</code>	Customer-facing app

Scope	Directory	Notes
admin	apps/admin	Internal dashboard
backend	packages/backend	Hono Worker, D1 migrations, cron
ui	packages/ui	Shared Radian-based components
types	packages/types	Shared TypeScript and zod types
api-client	packages/api-client	Typed fetch client used by the frontends
repo	Repository root	Turbo, Biome, tsconfig, pnpm, workflows, docs

When a change spans an app and the package it needs (a new referral form in `waitlist` plus a new endpoint in `backend`), either split it into two PRs or scope the PR to the user-visible side and explain the rest in the body. Splitting is usually better in a monorepo with separate deploys, because the backend change can ship first (section 6).

Breaking changes

Add `!` after the scope and a footer, for example `feat(backend)!: require verified email for referrals with BREAKING CHANGE: POST /signup now returns 409 for unverified referrers`. Because the Worker and the frontends deploy separately, a breaking API change should almost never be a single PR. Prefer adding the new behaviour, migrating callers, then removing the old behaviour.

What scopes do and do not do

Scopes make history readable, searchable (`git log --grep='^feat(waitlist)'`) and changelog-friendly. They do **not** decide what CI builds or deploys, and the setup in this document deliberately does not read commit messages for that purpose.

The reason is correctness. A `chore(ui)` commit changes `packages/ui`, which the `waitlist`, `landing`, `user app` and `admin` all import, so all four must rebuild. A scope cannot express that, and a mis-scoped commit would silently skip a deploy that was needed. Instead:

- **GitHub path filters** decide which deploy workflow starts on a push to `main` (section 5).

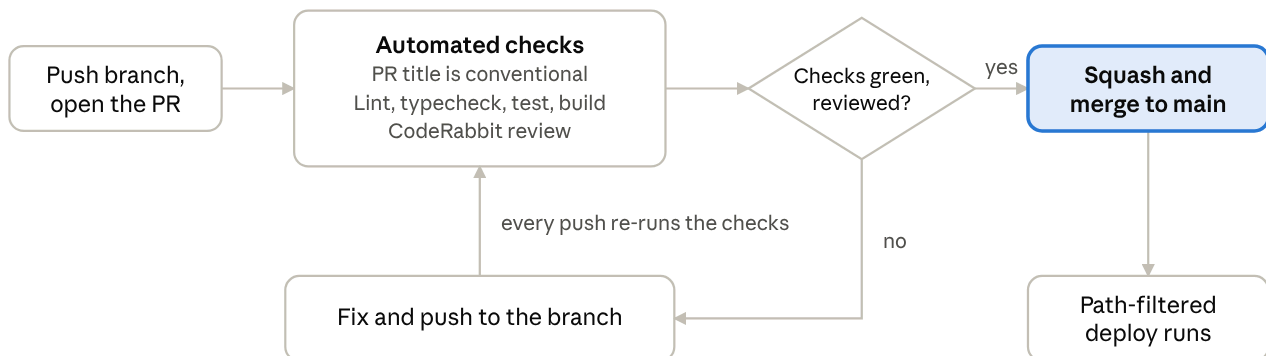
- **Turborepo's git-based filter** decides which packages to typecheck, test and build on a pull request, and follows the dependency graph, so a change in `packages/ui` selects every app that depends on it (section 3).
- **Turborepo's cache** skips work whose inputs have not changed, whatever the commit says.

That gives the saving in GitHub Actions minutes you want, driven by the files that changed rather than by the label someone typed.

3. Pull request workflow

All changes to `main` pass through a pull request, including solo work. The PR is where the automated checks, the review bot and the squash happen, and it leaves a record of why each change was made.

Nothing reaches main until checks and review pass



pull request lifecycle · 5 steps, 1 decision

A failed check or a review comment sends the author back to the branch; every push re-runs the checks, and only a merge to `main` can start a deploy.

Steps

1. **Open the PR.** Push the branch and open a PR against `main` with `gh pr create`. The **title** must be a valid Conventional Commit with a scope (section 2), for example `feat(waitlist): add referral code field`, because it becomes the commit message on `main`.

2. **Automated checks.** GitHub Actions starts two workflows: `PR title` (validates the title against the type and scope lists) and `CI` (lint, then typecheck, test and build for the affected packages only).
3. **Automated review.** CodeRabbit reads the diff and comments on bugs, security problems and logic errors in the Next.js and Hono code. It is advisory, so treat it as a fast first reviewer, not a gate (configuration in section 4).
4. **Human review.** Read the diff yourself, or ask someone else to. Resolve every comment from CodeRabbit by fixing it or replying why not.
5. **Squash and merge.** Once the required checks are green, merge with **Squash and merge**. All the small commits on the branch collapse into one clean commit on `main`, titled with the PR title. Delete the branch.

The CI workflow

The task names in this repo are `typecheck`, `test` and `build` (Turborepo), and `lint` is Biome run once at the root, so the check command is not `turbo run build lint check-types` but the pair below. The `...[origin/main]` filter means "every package that changed since `main`, plus everything that depends on one".

```
# .github/workflows/ci.yml
name: CI

# Runs on every PR. Deliberately NOT path-filtered: a required check that is
# skipped by a path filter stays "Pending" and blocks the merge forever.
# Turborepo does the selection instead, from the files that changed.
on:
  pull_request:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.event.pull_request.number }}
  cancel-in-progress: true

env:
  TURBO_TELEMETRY_DISABLED: "1"

jobs:
  verify:
```

```

name: Verify
runs-on: ubuntu-latest
timeout-minutes: 15
steps:
  - uses: actions/checkout@v7
    with:
      fetch-depth: 0 # turbo needs history to diff against the base
branch
  - uses: pnpm/action-setup@v6
  - uses: actions/setup-node@v7
    with:
      node-version: 22
      cache: pnpm
  - run: pnpm install --frozen-lockfile
  - name: Lint (Biome, whole repo, takes seconds)
    run: pnpm lint
  - name: Typecheck, test and build what this PR affects
    run: pnpm turbo run typecheck test build --filter="...[origin/${{
github.base_ref }}]"

```

Two details matter here. `fetch-depth: 0` is required because the default shallow checkout has no history for Turborepo to compare against. And a GitHub Actions runner starts with an empty Turborepo cache, so without the filter every PR would rebuild every app; the filter is what delivers the saving.

I checked the filter against this monorepo locally (a throwaway git repo with the real packages), comparing a branch to `main`:

What the PR changes	Packages Turborepo selects
<code>packages/ui</code>	<code>ui</code> , <code>waitlist</code> , plus the upstream <code>types</code> and <code>api-client</code> builds that <code>waitlist</code> needs
<code>packages/backend</code>	<code>backend</code> , plus the upstream <code>types</code> build
<code>turbo.json</code> (any root config)	every package
Nothing in a package (for example only <code>README.md</code>)	no tasks

The upstream builds appear because `build` depends on `^build`; they are small. When `landing`, `user-app` and `admin` exist they follow the same rule: an app is selected when it, or a package it imports, changes. This was run with Turborepo's dry run on this repo; it has not yet run on a real GitHub pull request.

The PR title check

Squash and merge only produces a good history if the title is good, so a second small workflow validates it. It is separate from `CI` so that editing a title does not re-run the build.

```
# .github/workflows/pr-title.yml
name: PR title

on:
  pull_request:
    types: [opened, edited, synchronize, reopened]

permissions:
  pull-requests: read

jobs:
  conventional-title:
    name: Conventional title
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: amannn/action-semantic-pull-request@v6
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        with:
          types: |
            feat
            fix
            perf
            refactor
            chore
            docs
            test
            ci
          scopes: |
            waitlist
```

```

    landing
    user-app
    admin
    backend
    ui
    types
    api-client
    repo
  requireScope: true
  subjectPattern: ^[a-z].+[^.]+$
  subjectPatternError: |
    The subject "{subject}" must start with a lower-case letter and
    must not end with a full stop.

```

When you add a new app or package, add its name to `scopes` in the same PR.

4. Repository configuration

Three things have to be set up once: the root `.gitignore`, the GitHub repository settings that make the workflow enforceable, and CodeRabbit.

`.gitignore`

Patterns without a leading slash match at any depth, so one root file covers every app and package. This version keeps your additions and fixes four things in the draft:

- `d1/migrations/.wrangler` is not a path in this repo. Local D1 state lives in `.wrangler` inside `packages/backend`, which the plain `.wrangler` pattern already covers.
- `node_modules` sat under the Cloudflare heading; it now has its own.
- `.dev.vars` (the Worker's local secrets file), `dist` and `*.tsbuildinfo` were missing.
- **SQL migrations must be committed.** They live in `packages/backend/migrations`, and nothing here ignores them. Only the generated local database state is ignored.

```

# Dependencies
node_modules

# Turborepo
.turbo

# Next.js (the static export is written to out/)
.next

```

```

out

# Build output
dist
*.tsbuildinfo

# Cloudflare and Wrangler local state (local D1, R2 and KV live under
.wrangler)
.wrangler
.dev.vars

# Environment secrets
.env
.env.local
.env*.local

# Editors and OS
.DS_Store

```

I tested this file with `git check-ignore` in the throwaway monorepo: the local D1 state, `out`, `.next`, `dist`, `.env*.local`, `.dev.vars` and `.turbo` are ignored, while `.env.example`, `.dev.vars.example`, the migrations, `turbo.json` and `pnpm-lock.yaml` are tracked. Commit the lockfile: CI installs with `--frozen-lockfile`.

GitHub repository settings

Set these under **Settings** → **General** → **Pull Requests**:

- Allow squash merging, and turn off merge commits and rebase merging, so squash is the only option.
- Squash commit message: **Pull request title** for the title and **Pull request description** for the body. The default of "commit or PR title" would use the first commit's message on single-commit PRs and defeat the title check.
- Automatically delete head branches.

The same settings from the command line (replace `OWNER/REPO`):

```
gh repo edit OWNER/REPO \
  --enable-squash-merge \
  --enable-merge-commit=false \
  --enable-rebase-merge=false \
  --delete-branch-on-merge

gh api -X PATCH repos/OWNER/REPO \
  -f squash_merge_commit_title=PR_TITLE \
  -f squash_merge_commit_message=PR_BODY
```

Then protect `main` with a branch ruleset (**Settings** → **Rules** → **Rulesets** → **New branch ruleset**, target `main`):

Rule	Setting	Why
Require a pull request before merging	On, 0 required approvals for a solo repo (raise it when you add collaborators)	GitHub does not let you approve your own PR, so requiring 1 would lock you out
Require status checks to pass	<code>Verify</code> and <code>Conventional title</code> , with <i>Require branches to be up to date on</i>	These are the job names from section 3
Block force pushes and deletions	On	<code>main</code> is production
Require linear history	On	Matches squash-only merging

Do **not** add CodeRabbit as a required check: it can be slow or skipped, and a required check that never reports blocks the merge.

The deploy jobs use a GitHub **environment** named `production`. Create it under **Settings** → **Environments** and store `CLOUDFLARE_API_TOKEN` and `CLOUDFLARE_ACCOUNT_ID` there (or as repository secrets, as in the technical setup document). Environment secrets are only released to jobs that run for that environment, which is what the deploy workflows in section 5 use. I have not run these settings against a live GitHub repository.

CodeRabbit

Install the CodeRabbit GitHub app on the repository, then add this file at the repo root. The keys below match CodeRabbit's configuration reference; the instructions are specific to this stack so reviews stay useful.

```
# .coderabbit.yaml
language: "en-US"
reviews:
  profile: "chill"
  high_level_summary: true
  request_changes_workflow: false
  auto_review:
    enabled: true
    drafts: false
  path_filters:
    - "!**/pnpm-lock.yaml"
    - "!**/out/**"
    - "!**/.next/**"
    - "!**/dist/**"
  path_instructions:
    - path: "apps/**"
      instructions: >-
        Next.js static export (output: export). Flag anything that needs a
server runtime:
        server actions, route handlers, cookies() or headers(), next/image
optimisation.
        Flag any NEXT_PUBLIC_ variable that holds a secret.
    - path: "packages/backend/src/**"
      instructions: >-
        Hono on Cloudflare Workers with D1. Check that every route validates
input with zod,
        SQL is parameterised (never string-built), CORS stays an allow-list,
secrets are never
        logged, and D1 writes are kept to the minimum because of the free-
tier quota.
    - path: "packages/backend/migrations/**"
      instructions: >-
        Migrations must work with both the previous and the new Worker code.
Flag any DROP,
        RENAME or NOT NULL addition on a column the live Worker still reads
or writes.
    - path: "packages/ui/**"
      instructions: >-
        Shared Radian components on Tailwind and Radix. Check accessibility
and keep
        app-specific logic out of the shared package.
    - path: ".github/workflows/**"
```

```
instructions: >-
  Check least-privilege permissions, that secrets are never echoed, and
  that every
  deploy workflow's path filter lists all of its dependencies.
```

Cost. CodeRabbit is paid per developer for private repositories, with a free trial; at the time of writing the plans start at roughly \$24 per developer per month billed annually, and CodeRabbit also advertises an open-source plan, so check whether you qualify. Check the [pricing page](#) before relying on it, because this project is meant to stay on free tiers. Nothing else in this workflow depends on CodeRabbit, so if you skip it, delete step 3 of section 3 and carry on.

5. Path-filtered deployments

On a push to `main`, each deployable unit has its own workflow with a `paths` filter, so changing the admin dashboard never redeploys the waitlist. The filter is a list of **everything that can change what that unit ships**: its own directory, every workspace package it imports, and the root files that affect a build.

Dependency map

This is the real import graph of the repo today (from each `package.json`). `landing`, `user-app` and `admin` will look like `waitlist`.

Deploy workflow	Own directory	Workspace packages it imports	Root files
<code>deploy-waitlist.yml</code>	<code>apps/waitlist/**</code>	<code>packages/ui/**</code> , <code>packages/api-client/**</code> , <code>packages/types/**</code>	lockfile, workspace, tsconfig, turbo, root <code>package.json</code> , its workflow files
<code>deploy-backend.yml</code>	<code>packages/backend/**</code>	<code>packages/types/**</code>	same root files, its own workflow file

Two entries are not in the draft. `packages/types/**` is needed because `api-client` and `backend` both import it, so a change there changes both outputs. The lockfile and `tsconfig.base.json` are needed because a dependency bump or a compiler setting changes the build without touching any app directory. The root `package.json` pattern has no `**`, so it matches only the root file, not every `package.json` in the repo.

Path filters are intentionally a little coarse (a lockfile change redeploys every app). That is the safe direction: the worst case is one unnecessary deploy, never a missed one.

Reusable frontend workflow

All frontends deploy the same way, so the steps live once in a reusable workflow and each app's file only supplies its name and its paths.

```
# .github/workflows/_deploy-frontend.yml
name: Deploy frontend (reusable)

on:
  workflow_call:
    inputs:
      app:
        description: Directory name under apps/ (for example waitlist)
        type: string
        required: true

permissions:
  contents: read

env:
  TURBO_TELEMETRY_DISABLED: "1"

jobs:
  deploy:
    name: Deploy ${inputs.app}
    runs-on: ubuntu-latest
    timeout-minutes: 10
    environment: production
    steps:
      - uses: actions/checkout@v7
      - uses: pnpm/action-setup@v6
      - uses: actions/setup-node@v7
        with:
          node-version: 22
          cache: pnpm
      - run: pnpm install --frozen-lockfile
      - name: Build static export
        run: pnpm turbo run build --filter=@matr/${inputs.app}
      - name: Deploy static assets to Cloudflare
        uses: cloudflare/wrangler-action@v4
```

```

with:
  apiToken: ${{ secrets.CLOUDFLARE_API_TOKEN }}
  accountId: ${{ secrets.CLOUDFLARE_ACCOUNT_ID }}
  workingDirectory: apps/${{ inputs.app }}
  command: deploy

```

The `deploy` command publishes the static export as a Cloudflare Workers assets-only Worker, as set up in the technical setup document. If you keep a Pages project instead, replace the last step's command with `pages deploy out --project-name <project>`.

Per-app workflow: waitlist

This is your `deploy-waitlist.yml`, completed.

```

# .github/workflows/deploy-waitlist.yml
name: Deploy waitlist

on:
  push:
    branches: [main]
    paths:
      - "apps/waitlist/**"
      - "packages/ui/**"
      - "packages/api-client/**"
      - "packages/types/**" # api-client and the app import it
      - "pnpm-lock.yaml"
      - "pnpm-workspace.yaml"
      - "tsconfig.base.json"
      - "turbo.json"
      - "package.json"
      - ".github/workflows/deploy-waitlist.yml"
      - ".github/workflows/_deploy-frontend.yml"
    workflow_dispatch: # manual redeploy from the Actions tab

# One deploy at a time per app; never cancel a deploy that is already
# running.
concurrency:
  group: deploy-waitlist
  cancel-in-progress: false

permissions:
  contents: read

```

```
jobs:
  deploy:
    uses: ../github/workflows/_deploy-frontend.yml
    with:
      app: waitlist
    secrets: inherit
```

Backend workflow

The Worker deploys on its own paths and applies D1 migrations first. Migrations are written to be backward compatible (section 6), so the Worker that is still live keeps working while the new one replaces it.

```
# .github/workflows/deploy-backend.yml
name: Deploy backend

on:
  push:
    branches: [main]
    paths:
      - "packages/backend/**"
      - "packages/types/**"
      - "pnpm-lock.yaml"
      - "pnpm-workspace.yaml"
      - "tsconfig.base.json"
      - "turbo.json"
      - "package.json"
      - ".github/workflows/deploy-backend.yml"
    workflow_dispatch:

concurrency:
  group: deploy-backend
  cancel-in-progress: false

permissions:
  contents: read

env:
  TURBO_TELEMETRY_DISABLED: "1"

jobs:
  deploy:
```

```

name: Deploy API Worker
runs-on: ubuntu-latest
timeout-minutes: 10
environment: production
steps:
  - uses: actions/checkout@v7
  - uses: pnpm/action-setup@v6
  - uses: actions/setup-node@v7
    with:
      node-version: 22
      cache: pnpm
  - run: pnpm install --frozen-lockfile
  - name: Apply D1 migrations
    uses: cloudflare/wrangler-action@v4
    with:
      apiToken: ${ secrets.CLOUDFLARE_API_TOKEN }
      accountId: ${ secrets.CLOUDFLARE_ACCOUNT_ID }
      workingDirectory: packages/backend
      command: d1 migrations apply matr-d1 --remote
  - name: Deploy Worker
    uses: cloudflare/wrangler-action@v4
    with:
      apiToken: ${ secrets.CLOUDFLARE_API_TOKEN }
      accountId: ${ secrets.CLOUDFLARE_ACCOUNT_ID }
      workingDirectory: packages/backend
      command: deploy

```

These three files replace the single `ci-deploy.yml` from the technical setup document. Delete that file when you add them, otherwise every push to `main` deploys twice.

How the pieces fit

- **Pull requests** run `CI` and `PR title` on every PR, with no path filter, and Turborepo narrows the work (section 3). Those are the required checks.
- **Pushes to `main`** run only the deploy workflows whose paths changed. They are not required checks, so a skipped one cannot block anything.
- **Squash merging** helps here: each PR lands as one commit, so the changed-file list that GitHub tests against the filters is exactly that PR's files.
- **Order between backend and frontend.** If one PR touches both, the two workflows start at the same time and finish in either order. That is safe only because API changes are additive (section 6). If you ever need strict ordering, ship the backend change as its

own PR first.

Adding a new app

1. Create `apps/<name>` with its own `wrangler.toml` and a scope entry for it (`scopes` in `pr-title.yml` , the scope table in section 2, and `path_instructions` in `.coderabbit.yml` if it needs special rules).
2. Copy `deploy-waitlist.yml` to `deploy-<name>.yml` , change `app` , the concurrency group, the first path and the workflow file names, and list the workspace packages the new app imports.
3. Nothing changes in `ci.yml` : Turborepo picks the new app up from the workspace graph.

I parsed all five workflow files as YAML and ran the Turborepo filters locally. They have not yet run on GitHub, so expect to fix small things on the first run (an action version tag, a secret name), and use `workflow_dispatch` to retry a deploy without a code change.

6. Releasing safely from main

In this workflow a merge is a deploy, so there is no separate release step to catch mistakes. Safety comes from three habits: every deploy can be undone quickly, the database schema and the code never need to change at the same instant, and unfinished features ship switched off.

Rolling back

Pick the fix by how urgent it is.

Situation	Action	Notes
Production is broken now	<code>wrangler rollback</code> in the affected directory	Instant: points traffic at the previous deployment. Works for the API Worker (<code>packages/backend</code>) and for an app (<code>apps/waitlist</code>).

Situation	Action	Notes
Any bad change, after the fire is out	Revert PR: <code>git revert <squash-commit></code> , open a PR titled <code>revert(scope): ...</code> , merge	Leaves a record in history and redeploys through the normal path. A rollback alone is undone by the next unrelated deploy, so always follow it with a revert.
Bad data written	D1 Time Travel restore	Restores the database to an earlier point in time (7 days on the free plan).

```
# see recent deployments, then roll back to the previous one
cd packages/backend
npx wrangler deployments list
npx wrangler rollback

# data: inspect the restore points, then restore (this overwrites the live database)
npx wrangler d1 time-travel info matr-d1
npx wrangler d1 time-travel restore matr-d1 --timestamp=2026-10-07T12:00:00Z
```

A code rollback does not undo a database migration, and a Time Travel restore also discards every signup made after the restore point. That is why the next section exists. Use `revert` as the commit type for revert PRs, and add it to the `types` list in `pr-title.yml` if you want the title check to accept it.

Backward-compatible migrations (expand and contract)

The migration runs just before the new Worker goes live, and a rollback puts the old Worker back against the new schema. So the schema at any moment must work with both the previous and the next version of the code. The pattern is to change the database in three separate releases:

1. **Expand.** Add what the new code needs without removing anything: a new nullable column, a new table, a new index. Ship it alone.
2. **Migrate.** Deploy code that writes the new shape, then backfill existing rows.
3. **Contract.** Only after no deployed code reads the old shape, remove it in a later release.

Example: tracking when a subscriber verified their email. The `waitlist_subscribers` table already exists, so the first release adds a nullable column and nothing else:

```
-- packages/backend/migrations/0003_add_verified_at.sql (release 1: expand)
ALTER TABLE waitlist_subscribers ADD COLUMN verified_at INTEGER;
```

```
-- release 2, after the Worker writes verified_at: backfill old rows
UPDATE waitlist_subscribers
SET verified_at = updated_at
WHERE status = 'verified' AND verified_at IS NULL;
```

I ran this against the real migration history in a local SQLite database: the existing row simply has `verified_at` empty, so the old Worker is unaffected, and a `NOT NULL` column with no default is rejected outright, which is the failure to avoid. D1 is SQLite underneath, but I did not apply this to a remote D1 database.

Do in the same release as the code	Do in its own release
Add a table, a nullable column, or a column with a default	Add <code>NOT NULL</code> to an existing column
Add an index	<code>DROP COLUMN</code> , <code>DROP TABLE</code> , or <code>RENAME</code> anything the live Worker uses
Make a read tolerate a missing value	Change what a column means

To rename a column, expand (add the new one), migrate (write both, backfill, switch reads), then contract (drop the old one). Migrations are also a quota concern: every row an `UPDATE` touches counts as a D1 row written, so on a large table do a backfill in batches (see the quota strategy in the technical setup document).

Feature flags

A flag lets an unfinished feature merge to `main` safely. The simplest flag for this stack is a build-time variable: Next.js inlines `NEXT_PUBLIC_*` values into the static export, and the `build` task in `turbo.json` already lists `NEXT_PUBLIC_*` in its `env`, so changing a flag changes the cache key and forces a fresh build.

```
// apps/waitlist/lib/flags.ts
export const flags = {
  referrals: process.env.NEXT_PUBLIC_FEATURE_REFERRALS === "true",
} as const;
```

```
{flags.referrals && <ReferralCodeField />}
```

To flip it, set a GitHub repository variable `NEXT_PUBLIC_FEATURE_REFERRALS`, pass it to the build step with `env: NEXT_PUBLIC_FEATURE_REFERRALS: ${{ vars.NEXT_PUBLIC_FEATURE_REFERRALS }}` in `_deploy-frontend.yml`, and redeploy from the Actions tab with `workflow_dispatch`. Because it is a static export, the flag changes only on a redeploy, which is the trade-off for serving pages with no server and no extra requests. A runtime flag served by the API would flip instantly but costs an extra request and a D1 or cache read per visit, which is better avoided on the free tier.

Flags are temporary. Once the feature is on for everyone, remove the flag in a `chore` PR so dead branches do not accumulate.

Putting it together: a feature that spans API and UI

Take the referral program from the branching examples. Each step is its own short branch and PR, merged in this order:

1. `chore/d1-referral-columns`: expand the schema (`chore(backend): add referral columns`).
2. `feat/api-referrals`: additive endpoint, old clients unaffected (`feat(backend): add referral lookup endpoint`).
3. `feat/waitlist-referrals`: UI behind `NEXT_PUBLIC_FEATURE_REFERRALS` (`feat(waitlist): add referral code field`).
4. Flip the flag, then check the live site and `/health`.
5. `chore/remove-referrals-flag`: delete the flag and, much later, contract the schema if anything old remains.

If any step misbehaves, `wrangler rollback` takes it back in seconds, and because every step was additive, the other steps keep working.